



MODUL 9 · DATABASE DENGAN PRISMA 7

Driver Adapter & Prisma Client Singleton



@prisma/adapter-pg + singleton pattern = koneksi DB yang aman dan efisien di Next.js.

Kenapa Driver Adapter?

Prisma 7 tidak lagi bundel built-in database driver. Kita pilih sendiri.

Prisma 6 (ada built-in driver)

```
// Prisma 6: tidak perlu install driver terpisah
// Driver Rust ada di dalam binary Prisma

// schema.prisma
// datasource db {
//   provider = "postgresql"
//   url      = env("DATABASE_URL")
// }

// lib/db.ts - simple
import { PrismaClient } from '@prisma/client'
const prisma = new PrismaClient()
export { prisma }

// MASALAH:
// - Binary Rust = ~14MB bundle
// - Tidak kompatibel semua environment
// - Cold start lambat di serverless
// - Tidak bisa pakai PostgreSQL features
//   yang di-support pg tapi tidak Rust
```

Prisma 7 - driver adapter eksplisit

```
// Prisma 7: install pg + adapter terpisah
// bun add @prisma/adapter-pg pg

import { PrismaClient } from
import { PrismaPg } from '@prisma/adapter-pg'

// ✅ Buat adapter dengan connection string
const adapter = new PrismaPg({
  connectionString: process.env.DATABASE_URL!,
})

// ✅ Pass adapter ke PrismaClient
const prisma = new PrismaClient({ adapter })

// Keuntungan:
// - Bundle 90% lebih kecil (hanya JS)
// - Support semua pg features
// - Bisa pakai pg connection pool
// - Deploy ke Vercel/Cloudflare/Bun
```

Singleton Pattern — Satu Instance, Satu Connection Pool

Hot reload Next.js dev server bisa membuat terlalu banyak koneksi. Singleton mencegah ini.

lib/db.ts — Prisma 7 singleton yang benar

```
import { PrismaClient } from "../generated/prisma/client"
import { PrismaPg } from "@prisma/adapter-pg"

// — Singleton untuk mencegah connection exhaustion —
// Masalah: Next.js dev server hot-reload berkali-kali
// Tanpa singleton: setiap reload = PrismaClient baru = koneksi baru
// Dengan singleton: reuse instance yang sama

const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined
}

function createPrismaClient(): PrismaClient {
  const adapter = new PrismaPg({
    connectionString: process.env.DATABASE_URL!,
  })
  return new PrismaClient({
    adapter,
    log: process.env.NODE_ENV === "development"
      ? ["query", "error", "warn"]
      : ["error"],
  })
}

// Reuse di development, buat baru di production
export const db =
  globalForPrisma.prisma ?? createPrismaClient()

if (process.env.NODE_ENV !== "production") {
  globalForPrisma.prisma = db
}
```

P R I S M A 7 & P O S T G R E S Q L

Cara Pakai db di Server Component

Import db dari lib/db.ts di setiap Server Component yang butuh akses database.

Menggunakan db di berbagai konteks

```
import { db } from "@lib/db"

// — Di Server Component (paling umum) —————
export default async function UsersPage() {
  const users = await db.user.findMany({
    orderBy: { createdAt: "desc" },
  })
  return <UserList users={users} />
}

// — Di Server Action —————
export async function createUser(formData: FormData) {
  "use server"
  const user = await db.user.create({
    data: {
      email: formData.get("email") as string,
      name: formData.get("name") as string,
    },
  })
  return user
}

// — Di Route Handler (API) —————
// app/api/users/route.ts
export async function GET() {
  const users = await db.user.findMany()
  return Response.json(users)
}

// ⚠️ JANGAN import db di Client Component!
// db = Node.js package, tidak bisa di browser.
```

Evaluasi Pemahaman

Jawab sebelum lanjut.

Q1 — Mengapa Prisma 7 memerlukan `@prisma/adapter-pg` terpisah?

- A) Prisma 7 tidak support PostgreSQL tanpanya B) Hanya untuk production deployment C) Prisma 7 menghapus built-in Rust driver — kita pilih sendiri driver JavaScript yang dipakai

Q2 — Apa masalah yang diselesaikan oleh singleton pattern di Next.js development?

- A) Mencegah pembuatan terlalu banyak koneksi akibat hot-reload berkali-kali B) Mempercepat query database C) Memungkinkan akses db dari Client Component

Q3 — Apakah db (dari `lib/db.ts`) boleh diimport di Client Component?

- A) Ya, tapi hanya untuk read-only B) Tidak — pg adalah Node.js package yang tidak tersedia di browser bundle C) Ya, dengan 'use client' directive

Tugas Mandiri

Setup PrismaClient singleton yang benar untuk Next.js.

01 Install Adapter

`bun add @prisma/adapter-pg pg`. Verifikasi keduanya ada di `package.json dependencies`.

02 `lib/db.ts`

Buat singleton pattern lengkap dengan `globalForPrisma`, `createPrismaClient`, `PrismaPg` adapter. Import dari generated path.

03 Test Import

Import `db` di `app/page.tsx` (Server Component). Query: `db.user.count()`. Render count. Pastikan tidak ada TypeScript error.

04 Test Dev Reload

Dengan `bun run dev`, edit `page.tsx` dan save berkali-kali. Lihat logs terminal: Prisma seharusnya tidak buat connection baru setiap reload.

⚠ Import path: `import { PrismaClient } from '../generated/prisma/client'` — bukan `'@prisma/client'`!

Yang Sudah Kita Pelajari

- ✓ `@prisma/adapter-pg`: driver PostgreSQL eksplisit untuk Prisma 7. Wajib install.
- ✓ `PrismaPg({ connectionString })` → `new PrismaClient({ adapter })` — dua langkah.
- ✓ Singleton via `globalThis` mencegah connection exhaustion saat hot-reload development.
- ✓ Import db dari `@/lib/db` — bukan `@prisma/client` langsung.
- ✓ db TIDAK boleh diimport di Client Component — Node.js only package.

→ Selanjutnya: **Bab 5 — Migrations: migrate dev vs migrate deploy**

